Evaluating the Impact of Data Cleaning on k-Nearest Neighbors Performance

Ong, John Laurence E.
College of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
johnlaurence.ong@my.jru.edu

***Abstract*— The k-Nearest Neighbors (kNN) algorithm is a simple yet effective machine learning technique that is widely used for decision-making due to its lazy learning nature and easy implementation. This study explores the performance of the kNN algorithm using the Iris dataset, a standard benchmark introduced by Ronald A. Fisher in1936. The objective of this work is to predict Iris flower species based on sepal and petal measurement and to analyze how different parameters affect the model performance. The methodology involves organizing the implementation into five modules, including setup & imports, data acquisition, train-test and model building, prediction and evaluation metrics, and visualization. The Experimental analysis was conducted by modifying the number of neighbors (k), test_size, and random_state to observe the changes in accuracy and confusion matrix. Results show that the higher the value of k leads to more stable and robust predictions, while lower values increase sensitivity to noise and overfitting. The study also discusses real-world government applications of kNN and addresses ethical concerns related to transparency and decision support.**

Keywords—k-Nearest Neighbors, Iris Dataset, Classification, Machine Learning

I. Introduction

In many real world scenarios, predictive models are developed using datasets that are incomplete, noisy, or inconsistent data. These inconsistencies can significantly impact the performance of the model. The k-Nearest Neighbors (kNN) relies heavily on the consistency of data, and scale. Without proper data preparations, meaningful patterns will not be discovered which leads to inaccurate and unreliable predictions.

Data cleaning and preparation plays a crucial role in ensuring that the machine learning (ML) models can effectively learn from the data. By addressing the issues such as missing values (NaN), inconsistent categorical labels, and unscaled numerical features, the prediction of the model can leverage or improve its performance.

In this activity, the objective is to develop a k-Neareast Neighbor model to predict the Purchase_Likelihood based on the other available column within the excel and to evaluate the impact of the systematic data preparation on model performance by comparing the baseline model trained on raw data with an optimized model trained on a cleaned dataset. Moreover, this work will illustrate the effect or impact of different k values parameters to identify the highest or peak of the classification accuracy.

II. Methodology

This presents the organization and logical structure of the three given codes. The program then was divided into three modular components with sub components. Module 1 - Preprocessing, includes: Import and Load Datasets, Data Cleaning and Missing Value Handling, Feature Standardization, Engineering and Enrichment, and Scaling, Encoding, and Final Output. Module 2 - Preparation, includes: Import and Load Datasets, Baseline Data Preparation and Feature Target Separation, Train-Test Split and Model Training, lastly Model Evaluation. Module 3 - Model Training, includes: Import and Load Datasets, Handling Missing Values, and Split - Train, Predict and Evaluate.

A. Module 1 - Preprocessing

The module 1 which is the Preprocessing was all about the implementation of how you will clean the dataset. This will improve the dataset quality creating a more accurate prediction. The pipeline consisted of the following steps:

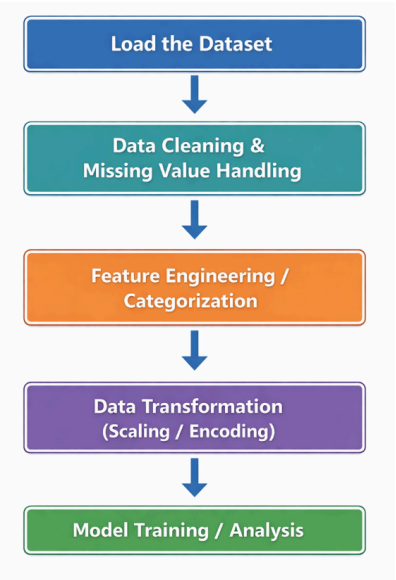


Fig. 1. Preprocessing Steps

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

1. Import and Load Datasets - These are used to initialize all the required or utilized libraries for data processing and machine learning. Afterwards, it will load the provided datasets.
2. Data Cleaning and Handling of Missing Values - Missing numerical will be replaced with the computed median values per column. It depends on the code if you want to remove or replace the value to reduce the noise.
3. Feature Standardization, Engineering and Enrichment - This is the way you categorize variables transforming into a numerical representation. Resulting for inconsistencies to be eliminated.
4. Scaling, Encoding, and Final Output - All the strings will be transformed into numerical values which will result in a normalization and consistent range of data, preventing a bias feature of model training.

B. Module 2 - Preparation

The module 2 which is the Preparation was all about ensuring that the final dataset used for model training are fully numeric and no missing values. Any rows that have NaN values will be removed to guarantee completeness. This step produces a clean and machine learning ready suitable for evaluation.

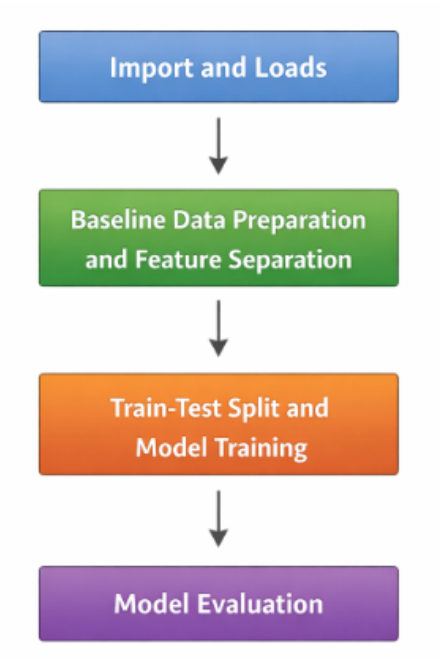
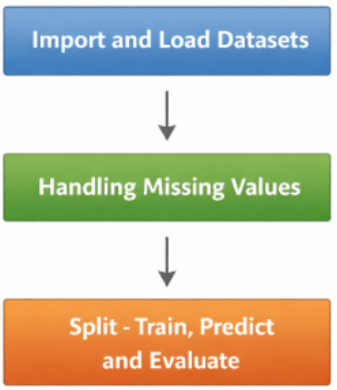


Fig. 2. Preparation Steps

1. Import and Load Datasets - These are used to initialize all the required or utilized libraries for data processing and machine learning. Afterwards, it will load the provided datasets.
2. Baseline Data Preparation and Feature Target Separation - This creates a simplified version of the dataset by keeping only the numeric values and removing any rows that have missing values. Afterwards, it will split into features and target variables.
3. Train-Test Split and Model Training - This is the actual training of the kNN model based on its hyperparameter.
4. Model Evaluation - This shows how well you trained the kNN model on the unseen data.

C. Module 3 - Model Training

This module 3 which is the Model Training



1. Import and Load Datasets - These are used to initialize all the required or utilized libraries for data processing and machine learning. Afterwards, it will load the provided datasets.
2. Handling Missing Values - Any remaining values were addressed to prevent errors during model training.
3. Split - Train, Predict and Evaluate - It begins to identify the target variable (Purchase_Likelihood_, then separate ir from the feature set. The data set then split into training and testing to evaluate accuracy.

D. Experimental Setup

To evaluate the effects of the data preparation from the pipeline which is the data cleaning, handling missing values, categorization, and scaling on the model performance, two experiments were performed using the k-Nearest Neighbors. All the experiments used the same train-test split 0.2 (20%), 80% training and 20% testing to ensure no bias predictions.

1. Baseline Run - The baseline preparation starts with the loading of the raw datasets which are the adjusted_noisy_knn_300.csv which are prone to noise. This baseline will keep only the numeric value, automatically dropping string values using the .dropna() function. Separation of features (x) and the target (y), the target value is the (Purchase_Likelihood) and the predictors are the other available columns. Split and Train the data with the k set into (5). The baseline model achieved an accuracy of 42.86%, which depicts that the raw dataset is prone to noise.

```
Baseline Accuracy: 42.86%
Baseline Confusion Matrix:
 [[3 1]
 [7 3]]
```

Fig. 4. Baseline Accuracy

2. Optimized Run - The optimized run utilizes the cleaned dataset which is generated from the pipeline. Wherein, all the values are numeric and no missing values. The k is set to (5) again to ensure a fair comparison of the model. The model result shows that the accuracy was raised from 42.68% to 72.41%, this indicates that the data cleaning process was effective since the model's accuracy increases drastically. The figure below shows the accuracy rate as well as the confusion matrix for more insights and information.

```
--- Optimized Run ---
Detecting 159 missing values...
Missing values handled.
Optimized Accuracy: 68.97%
Optimized Confusion Matrix:
 [[22  9]
 [ 9 18]]
Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.71      0.71        31
           1       0.67      0.67      0.67        27

    accuracy                           0.69        58
   macro avg       0.69      0.69      0.69        58
weighted avg       0.69      0.69      0.69        58
```

Fig. 5. Optimize Run Accuracy

III. Result and Experimental Analysis

This section shows the Data Quality Comparison, Performance comparison, and K-Optimization.

A. Data Quality Comparison

Before the data undergoes cleaning, the dataset that is provided is prone to noise. Inconsistent from the column Age and Purchase_Date, Outliers such as -10 and 200 values for Age, Redundancy from the columns Promo_User and Newsletter_Sub, Missingness data from the column Age and Total_Spend and Scale Imbalance for columns Annual Income and Support_Calls.

After applying different strategies and methods in the preparation pipeline, the dataset will be transformed into a more consistent numeric and structured organization of data. Creating more harmonized data drastically increases the dataset relevance and accuracy for kNN models.

B. Data Quality Comparison

This section shows the differences of the model's accuracy.

Model	Accuracy
Baseline (Raw)	42.86%
Cleaned	64.29%
Optimized (Modified)	68.97%

Table. 1. Difference of Model Accuracy

The table shown demonstrates a drastic change in accuracy percentage assuming that all instances (random_state and kNN values). As prone to noise, the baseline or raw has the least accuracy (42.86%) among the three, followed by a cleaned model (64.29%), which indicates that the preprocessing is successful. The Highest belongs to the Optimized model (68.97%), this showcases that the removal of other variables instead replacing them with median, creates more stability for the model. Indicating that the model is more stable and accurate. Moreover, the confusion matrix of the optimized run showed a better class separation and reduced misclassification rates.

C. Data Quality Comparison

```
--- k Parameter Optimization ---
k=1: Accuracy=67.24%
k=2: Accuracy=68.97%
k=3: Accuracy=72.41%
k=4: Accuracy=67.24%
k=5: Accuracy=68.97%
k=6: Accuracy=68.97%
k=7: Accuracy=68.97%
k=8: Accuracy=67.24%
k=9: Accuracy=65.52%
k=10: Accuracy=68.97%
k=11: Accuracy=68.97%
k=12: Accuracy=65.52%
k=13: Accuracy=67.24%
k=14: Accuracy=65.52%
k=15: Accuracy=68.97%
k=16: Accuracy=68.97%
k=17: Accuracy=68.97%
k=18: Accuracy=67.24%
k=19: Accuracy=68.97%
k=20: Accuracy=67.24%
```

Fig. 3. The variety of Accuracy at different k value

We are also tasked to evaluate the impact of kNN values to the accuracy of the model. The kNN model was evaluated from k values ranging from 1 to 20. The results showed that the peak of the k values is on k = 3, however, the accuracy declined as k became too large. This shows how important the experimentation of hyperparameters (k) to balance bias.

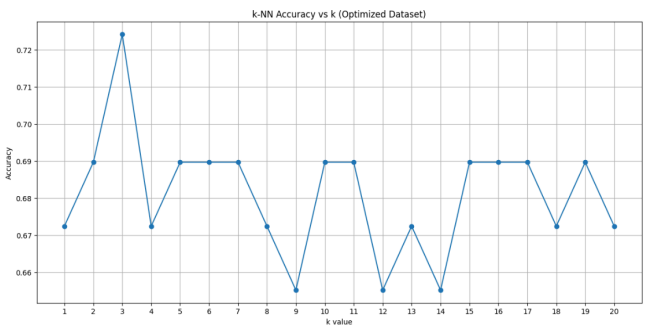


Fig. 6. Plotting of Accuracy based on k value

IV. CONCLUSION

One of the main highlights of this experiment was learning how the logic behind data cleaning actually works. As someone with no prior experience in cleaning data, understanding the overall flow of the cleaning process was overwhelming. At the beginning, I found it hard to comprehend all the functions present in the given code. However, as I progressed and worked through the notebook in Google Colab, I was gradually able to grasp the logic and process behind the data cleaning steps.

These are some of the key ideas that stood out to me. Preprocessing is where different strategies are applied to clean the dataset by modifying, removing, or categorizing data in order to create a better stability and consistency. Module 2, which is the preparation stage, ensures that all values are converted into numerical form. Any remaining string-based attributes are removed to make sure that no invalid or missing data enters the model. Module 3, which is the model training stage, is where the actual training of the model happens. This stage allows us to observe how effective the earlier preprocessing steps were.

Moreover, I was able to evaluate the model and observe that each preprocessing step contributed to an increase in accuracy, showing how important proper data preparation is. The results also showed that k = 3 produced the best performance, serving as a balanced value that helps reduce both overfitting and underfitting. As the k value increases, the accuracy gradually decreases, indicating that larger k values may oversmooth the decision boundaries and reduce model sensitivity.

Overall, this activity demonstrated that how well preprocessing is performed is crucial, as it serves as the foundation for model performance. A well-prepared dataset leads to more accurate and reliable predictions, while poor preprocessing can negatively impact the effectiveness of the model.